Lista Teórica 10 — Gabarito

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Aula 10 — KNN e Árvores de Classificação

Exercício 1 — o corte que o erro de classificação não enxerga. Um nó tem 400 observações, das quais 300 são da classe 1 — logo $p = 0{,}75$. Um corte candidato o divide em dois filhos de 200 observações cada:

	n	classe 1	classe 0
pai	400	300	100
esquerda	200	100	100
direita	200	200	0

As três medidas de impureza de um nó com proporção p são

$$\text{erro} = \min(p,\, 1-p), \qquad \text{Gini} = 2p(1-p), \qquad \text{entropia} = -p\log_2 p - (1-p)\log_2(1-p).$$

(a) Calcule a impureza do pai e a impureza ponderada dos filhos, pelas três medidas, e obtenha as três reduções.

(b) O corte é bom? (Olhe o filho da direita.) Qual das três medidas *não percebe* isso, e qual é exatamente a redução que ela indica?

(c) Que consequência isso tem para o algoritmo que **cresce** a árvore?

(d) Ainda assim, o erro de classificação é a medida recomendada para **podar**. Por quê?

Solução

(a) Proporções: pai $p = 0{,}75$, esquerda $p = 0{,}50$, direita $p = 1{,}00$. Os pesos são $200/400 = 1/2$ para cada filho.

medida	pai	filhos (ponderado)	redução
erro	$\min(0{,}75; 0{,}25) = 0{,}2500$	$\frac{1}{2}(0{,}50) + \frac{1}{2}(0) = 0{,}2500$	**0,0000**
Gini	$2(0{,}75)(0{,}25) = 0{,}3750$	$\frac{1}{2}(0{,}50) + \frac{1}{2}(0) = 0{,}2500$	**0,1250**
entropia	$0{,}8113$	$\frac{1}{2}(1{,}0000) + \frac{1}{2}(0) = 0{,}5000$	**0,3113**

(b) O corte é **claramente bom**: o filho da direita é *puro* — 200 observações, todas da classe 1. Separar 200 observações com certeza absoluta da classe é exatamente o que uma árvore deveria querer fazer.

A medida que não percebe é o **erro de classificação**, e a redução que ela indica é **exatamente zero**. Não é "pequena": é zero, na precisão de máquina. A razão é que, em ambos os filhos, a classe majoritária continua sendo a classe 1 — na esquerda por empate, na direita por unanimidade — e o erro de classificação só olha a classe majoritária de cada nó. Ele conta 100 erros no pai e $100 + 0 = 100$ erros nos filhos.

(c) Se a árvore usasse o erro de classificação como critério de corte, ela avaliaria este corte como **inútil** e possivelmente escolheria outro, ou pararia de crescer (se todos os cortes candidatos dessem redução zero, o que é comum). Um filho puro seria descartado.

É por isso que o `DecisionTreeClassifier` do `scikit-learn` oferece apenas `criterion="gini"` e `"entropy"` (ou `"log_loss"`) para crescer — **o erro de classificação não está entre as opções**. Gini e entropia são *estritamente* côncavas, e o Exercício 4 mostra que isso garante redução positiva sempre que os filhos diferirem.

(d) Porque podar e crescer respondem a perguntas diferentes.
Crescer é uma busca gulosa: precisa de um critério **sensível**, capaz de distinguir um corte bom de um corte medíocre mesmo quando nenhum dos dois muda a classe majoritária. Um critério que devolve zero com frequência não orienta a busca.
Podar é uma decisão sobre o **desempenho final** da árvore, e o desempenho final é medido pelo erro que ela vai cometer — que é, literalmente, o erro de classificação. Usar Gini para podar otimizaria uma quantidade que não é a que interessa ao usuário.
Em uma frase: *Gini e entropia são bons guias, o erro de classificação é o bom juiz.*

Exercício 2 — KNN nos extremos, em classificação.

(a) Qual é o erro de *treino* do KNN com $k = 1$, num conjunto sem pontos repetidos? E com $k = n$ (supondo n ímpar)?

(b) Descreva a fronteira de decisão nos dois extremos, em termos de viés e variância.

(c) Um resultado clássico (Cover e Hart, 1967) diz que, quando $n \to \infty$, o erro do classificador de 1 vizinho mais próximo, $R_{1\text{NN}}$, satisfaz

$$R^* \;\leq\; R_{1\text{NN}} \;\leq\; 2\,R^*,$$

onde R^* é o erro de Bayes. Se $R^* = 0{,}10$, em que intervalo está o erro assintótico do 1-NN? Comente: isso é uma boa ou uma má notícia sobre o 1-NN?

(d) Por que, na prática, o KNN com k escolhido por validação cruzada costuma bater o 1-NN, se o teorema já garante um fator 2?

Solução

(a) Com $k = 1$ o erro de treino é **exatamente zero**: o vizinho mais próximo de $\boldsymbol{X}_i$ é o próprio $\boldsymbol{X}_i$, a distância 0, então $\widehat{g}(\boldsymbol{X}_i) = Y_i$ sempre. Com $k = n$, todos os pontos são vizinhos de todos e o classificador devolve sempre a classe majoritária do conjunto inteiro; o erro de treino é a proporção da classe *minoritária*.

(b) Com $k = 1$ a fronteira é o contorno do diagrama de Voronoi entre pontos de classes diferentes: extremamente recortada, com ilhas em torno de observações isoladas. **Viés baixo, variância altíssima** — trocar uma observação muda a fronteira na vizinhança dela.
Com $k = n$ a fronteira *não existe*: o classificador é constante. **Viés máximo, variância zero**. É o análogo em classificação do preditor constante do Exercício 2 da Lista Teórica 01.

(c) Com $R^* = 0{,}10$, o erro assintótico do 1-NN está em $[0{,}10;\ 0{,}20]$.
É uma notícia **surpreendentemente boa**, e vale apreciar por quê: o 1-NN é o classificador mais ingênuo que se pode imaginar — ele não estima nada, não tem parâmetros, não faz suposição nenhuma sobre a distribuição — e mesmo assim, com dados suficientes, nunca erra mais que o *dobro* do classificador ótimo, que conhece as densidades. Não existe resultado análogo para a regressão linear ou para o LDA: se as suposições deles falharem, o erro pode ser arbitrariamente ruim.
A má notícia está na palavra *assintótico*. A garantia é real e o n que a torna operante, não.

(d) Por três razões que se somam.
Primeiro, o teorema é **assintótico**: com n finito o 1-NN está bem acima do seu próprio limite, porque a variância domina.
Segundo, o fator 2 é uma **cota**, e cotas costumam ser folgadas. Um k maior tipicamente chega bem mais perto de R^* do que de $2R^*$ — a média sobre k vizinhos reduz a variância por um fator próximo de k, ao preço de um viés que cresce devagar enquanto os vizinhos ainda estiverem próximos.
Terceiro, e mais importante na prática: o k ótimo depende de n, de p e do nível de ruído, e ninguém sabe qual é. A validação cruzada o encontra sem precisar conhecer nada disso — e, como a Lista prática 03 mediu, a curva de erro contra k costuma ser *plana* numa faixa larga, o que significa que basta acertar a ordem de grandeza.

Exercício 3 — o que cada classificador supõe. Complete a tabela abaixo para os classificadores vistos no curso.

classificador	suposição principal	forma da fronteira	gera./discr.
regressão logística			
LDA			
QDA			
naive Bayes gaussiano			
KNN			
árvore			
SVM linear			
SVM com RBF			

(a) Preencha as três colunas.

(b) Quais desses métodos produzem estimativas de $\mathbb{P}(Y = 1 \mid \boldsymbol{X} = \boldsymbol{x})$ diretamente? Quais delas você usaria numa conta de custo esperado, à luz da Aula 08?

(c) Um deles é *invariante* a transformações crescentes de cada covariável separadamente (trocar x_j por $\log x_j$, por exemplo). Qual é, e por quê? Que consequência prática isso tem?

Solução

(a)

classificador	suposição principal	fronteira	tipo
logística	$\log \frac{p}{1-p}$ é linear em $\boldsymbol{x}$; nada sobre a distribuição de $\boldsymbol{X}$	linear	discriminativo
LDA	$\boldsymbol{X} \mid Y = k$ gaussiana, covariância *comum*	linear	generativo
QDA	$\boldsymbol{X} \mid Y = k$ gaussiana, covariâncias distintas	quadrática	generativo
naive Bayes	$\boldsymbol{X} \mid Y = k$ gaussiana com covariância *diagonal*	quadrática	generativo
KNN	r varia devagar (Lipschitz); a métrica de distância faz sentido	arbitrária, recortada	discriminativo
árvore	r é aproximadamente constante em blocos *alinhados aos eixos*	escada ortogonal	discriminativo
SVM linear	fronteira linear; só os pontos da margem importam	linear	discriminativo
SVM com RBF	fronteira suave; similaridade decai com a distância euclidiana	suave, arbitrária	discriminativo

(b) Dão probabilidades **diretamente**: logística, LDA, QDA, naive Bayes e KNN (a proporção entre os k vizinhos). A árvore dá a proporção da folha. A **SVM não dá**: ela devolve uma distância com sinal, não uma probabilidade, e o `predict_proba` do `scikit-learn` só existe com `probability=True`, que ajusta *por cima* do modelo, com validação cruzada interna, uma regressão logística sobre essa distância (o método de Platt). Numa conta de custo esperado (Exercício 2 da Lista Teórica 08) o que importa não é ter um número entre 0 e 1, é ele **estimar bem** $\mathbb{P}(Y = 1 \mid \boldsymbol{X} = \boldsymbol{x})$, porque o corte ótimo da Proposição 3.1 da Aula 08 foi deduzido para as probabilidades verdadeiras. Entre os métodos da tabela, a logística é a que as estima com menos suposições: modela $\mathbb{P}(Y = 1 \mid \boldsymbol{X} = \boldsymbol{x})$ diretamente, sem supor nada sobre a distribuição de $\boldsymbol{X}$, enquanto LDA, QDA e naive Bayes só acertam a probabilidade se a forma suposta para $\boldsymbol{X} \mid Y$ estiver certa. O naive Bayes é o caso mais delicado: com covariáveis correlacionadas, ele conta a mesma evidência várias vezes e empurra as probabilidades para 0 e 1 (Aula 07). KNN e árvores dão probabilidades grosseiras, com granularidade $1/k$ e $1/n_{\text{folha}}$ respectivamente.

(c) É a **árvore**. (Fora da tabela, os *ensembles* de árvores herdam dela a mesma propriedade.)

A razão é que a árvore só usa as covariáveis por meio de comparações $x_j < c$. Se g é estritamente crescente, então $x_j < c \iff g(x_j) < g(c)$: o mesmo conjunto de partições continua disponível, e o corte ótimo separa as mesmas observações, com o limiar transformado. A árvore ajustada é a mesma: as mesmas folhas, com as mesmas observações de treino e as mesmas predições nelas. O único detalhe fica por conta da implementação: o `scikit-learn` põe cada limiar no meio do caminho entre os dois valores observados vizinhos, e o ponto médio não se transforma junto — um ponto *novo* que caia exatamente entre esses dois valores pode mudar de lado.

A consequência prática é grande: **árvores não precisam de padronização**, nem de correção de assimetria, nem de tratamento de escala. Compare com o LDA (a matriz de covariância muda), com a SVM com RBF (a distância euclidiana muda) ou com o KNN (idem) — todos precisam. É uma das razões de florestas e *boosting* serem tão populares em dados tabulares heterogêneos: eles dispensam metade do `Pipeline` da Aula 06.

(A ressalva: a invariância vale para transformações de *uma* covariável de cada vez. Rotacionar o espaço — misturar x_1 e x_2 — muda tudo, porque os cortes são obrigatoriamente alinhados aos eixos. É por isso que árvores vão mal em fronteiras diagonais, e é exatamente

a fragilidade oposta à do LDA.)

Exercício 4 — concavidade: por que Gini sempre ganha algo. Seja $\phi : [0,1] \to \mathbb{R}$ uma medida de impureza e considere um nó com proporção p dividido em dois filhos de pesos w e $1-w$ e proporções p_E e p_D, com

$$p = w\,p_E + (1-w)\,p_D$$

(a proporção do pai é sempre a média ponderada das dos filhos — confira isso no Exercício 1).

(a) Mostre que, se ϕ é **côncava**, a redução de impureza $\phi(p) - [w\,\phi(p_E) + (1-w)\,\phi(p_D)]$ é sempre ≥ 0.

(b) Mostre que, se ϕ é **estritamente** côncava, a redução é > 0 sempre que $p_E \neq p_D$.

(c) Verifique que $\phi(p) = \min(p, 1-p)$ é côncava mas **não** estritamente. Use isso para explicar, em uma linha, o resultado do Exercício 1.

(d) Mostre que o Gini $2p(1-p)$ é o **dobro da variância** de uma Bernoulli(p). Que interpretação isso dá à "redução de impureza de Gini"?

Solução

(a) É exatamente a desigualdade de Jensen. Por definição, ϕ côncava significa

$$\phi(w\,p_E + (1-w)\,p_D) \;\geq\; w\,\phi(p_E) + (1-w)\,\phi(p_D)$$

para todo $w \in [0,1]$. Como $p = w\,p_E + (1-w)\,p_D$, o lado esquerdo é $\phi(p)$, e a redução

$$\phi(p) - [w\,\phi(p_E) + (1-w)\,\phi(p_D)] \;\geq\; 0.$$

Ou seja: **cortar nunca aumenta a impureza**, qualquer que seja a medida côncava. É a garantia mínima que se pede de um critério de corte.

(b) Concavidade estrita é a mesma desigualdade com $>$ no lugar de $\geq$, desde que $p_E \neq p_D$ e $0 < w < 1$. Logo a redução é estritamente positiva nessas condições. Em palavras: **qualquer corte que separe os filhos em proporções diferentes é premiado**, mesmo que a classe majoritária não mude.

(c) A função $\min(p, 1-p)$ é linear por partes:

$$\min(p, 1-p) = \begin{cases} p, & 0 \leq p \leq \frac{1}{2}, \\ 1-p, & \frac{1}{2} \leq p \leq 1. \end{cases}$$

É côncava (o gráfico é um "$\wedge$" com vértice em $p = 1/2$), mas em cada uma das duas metades ela é *linear* — e função linear satisfaz Jensen com igualdade.
No Exercício 1, $p_E = 0{,}50$ e $p_D = 1{,}00$ estão **os dois no ramo** $[\frac{1}{2}, 1]$, onde $\phi(p) = 1-p$ é linear. Então a média ponderada de ϕ é igual a ϕ da média ponderada:

$$\tfrac{1}{2}(1-0{,}5) + \tfrac{1}{2}(1-1{,}0) = 0{,}25 = 1-0{,}75 = \phi(0{,}75),$$

e a redução é exatamente zero. Não foi coincidência da escolha dos números: é o que *sempre* acontece quando os dois filhos caem do mesmo lado de $1/2$.

(d) Se $Z \sim$ Bernoulli(p), então $\mathbb{E}[Z] = p$ e $\mathbb{E}[Z^2] = \mathbb{E}[Z] = p$ (porque $Z^2 = Z$), logo

$$\mathrm{Var}(Z) = p - p^2 = p(1-p) = \tfrac{1}{2} \cdot 2p(1-p) = \tfrac{1}{2}\,\mathrm{Gini}(p).$$

O índice de Gini é, portanto, o dobro da variância de uma Bernoulli — e, a menos desse fator 2, *é* a variância.

A interpretação é bonita e unifica o curso: a redução de impureza de Gini é exatamente a **redução da soma de quadrados dentro dos grupos** que a árvore de *regressão* da Aula 05 usa, aplicada à variável indicadora $\mathbf{1}\{Y = 1\}$. Crescer uma árvore de classificação por Gini é a mesma coisa que crescer uma árvore de regressão sobre o indicador da classe — o mesmo algoritmo, o mesmo critério, escrito de outro jeito.